

👁 Watch 0

🍴 Fork 0

☆

Qualcomm utilities for Linux builds

📄 BSD 3-Clause Clear License

👤 Contributing

☆ 0 stars 🍴 0 forks 👁 0 watching 👤 1 branch 🏷 0 tags ↻ Activity

🌐 Public repository · Forked from [qualcomm-linux/qcom-build-utils](#)

👤 m

🍴 1 Branch

🏷 0 Tags

👤

🔍 Go to file

T

Go to file

Add file +

Code

...

This branch is up to date with `qualcomm-linux/qcom-build-utils:main`.

👤 Contribute

↻ Sync fork

	rootfs: clean up local-debs repo and apt caches from final image	a4390b7 · 4 hours ago	🕒
📁 .github	pkg-release: add AXIOM_Check gate before ...	yesterday	
📁 bootloader	bootloader: add --seed-volatile-vars support ...	2 days ago	
📁 docs	release: enforce lintian in release build	2 weeks ago	
📁 flash	flash: support per-target VolatileVars.bin inj...	2 days ago	
📁 kernel	build-dtb-image: probe Debian standard DTB...	3 months ago	
📁 rootfs/scripts	rootfs: clean up local-debs repo and apt cac...	4 hours ago	
📁 scripts	promote: peel upstream ref to a commit	2 weeks ago	
📄 .gitignore	Add files for building ubuntu debs	last year	
📄 AGENTS.md	Factor branch parsing logic	last month	
📄 CONTRIBUTING.md	deb_abi_checker.py : When used from the c...	11 months ago	
📄 LICENSE.txt	Adding LICENCE, CONTRIBUTING and pr te...	11 months ago	
📄 README.md	Factor branch parsing logic	last month	

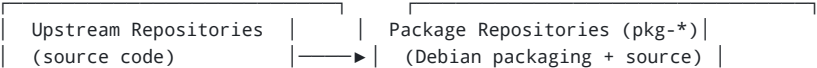
READMEContributingLicense

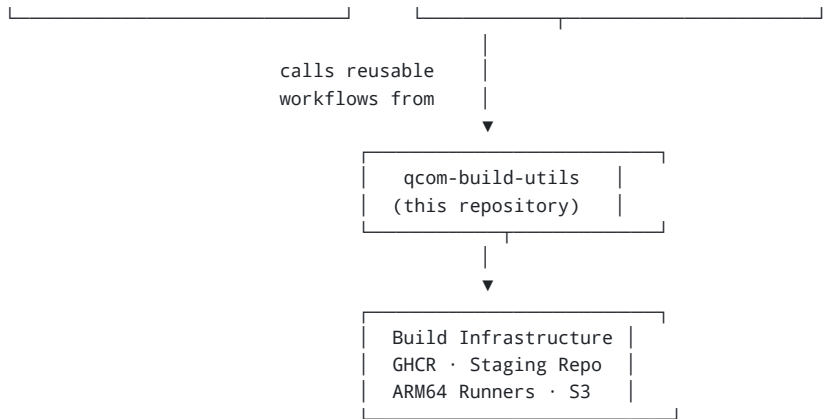
qcom-build-utils

Centralized build tooling, reusable GitHub workflows, and composite actions for the [Qualcomm Linux](#) Debian package ecosystem. This repository standardizes how `pkg-*` package repositories build, validate, promote, and release Debian packages for Qualcomm ARM64 platforms.

Architecture Overview

The Qualcomm Linux packaging system is composed of four main components:





Upstream Repositories contain the project source code (e.g., [qcom-example-package-source](#)). **Package Repositories** (prefixed `pkg-`) hold Debian packaging metadata, track upstream versions, and invoke the reusable workflows defined here. New package repos are created from the [pkg-template](#). A complete working example is available at [pkg-example](#).

Repository Structure

```
qcom-build-utils/
├── .github/
│   ├── actions/           # Composite GitHub Actions
│   │   ├── abi_checker/   # ABI compatibility checks
│   │   ├── build_package/ # Debian package build (gbp + sbuild)
│   │   └── push_to_repo/   # Publish packages to staging APT repo
│   └── workflows/         # Reusable workflow definitions
│       ├── pkg-build-reusable-workflow.yml
│       ├── pkg-promote-reusable-workflow.yml
│       ├── pkg-upstream-pr-build-reusable-workflow.yml
│       ├── pkg-release-reusable-workflow.yml
│       └── qcom-preflight-checks.yml
├── scripts/               # Python & shell build utilities
│   ├── deb_abi_checker.py  # ABI comparison tool (libabigail)
│   ├── ppa_interface.py    # APT repository interface
│   ├── ppa_organizer.py    # Build output organizer
│   ├── create_promotion_pr.py # PR generation for promotions
│   ├── merge_debian_packaging_upstream # Upstream merge script
│   └── helpers.py          # Shared utility functions
├── kernel/scripts/        # Kernel build scripts
│   ├── build_kernel.sh     # ARM64 kernel build
│   ├── build-kernel-deb.sh # Kernel .deb packaging
│   └── build-dtb-image.sh   # Device Tree Blob image builder
├── bootloader/
│   └── build-efi-esp.sh     # EFI System Partition builder
├── rootfs/scripts/
│   └── build-rootfs.sh     # Root filesystem image builder
└── docs/                  # Detailed documentation
```

Reusable Workflows

Package repositories call these workflows from their own `.github/workflows/` directory. Each workflow is invoked with `uses: qualcomm-linux/qcom-build-utils/.github/workflows/<workflow>@main`.

Workflow	Purpose
pkg-build-reusable-workflow	Main package build workflow — routes Debian suites through Debusine and Ubuntu codenames through the local pkg-builder path.
pkg-promote-reusable-workflow	Promotes a new upstream release into a package repo — merges upstream code, updates changelog, and creates a PR.
pkg-upstream-pr-build-reusable-workflow	Validates that PRs in an upstream repo won't break the Debian package build. Called from the upstream repo.

Workflow	Purpose
pkg-release-reusable-workflow	Triggers a formal release — Debian suites use Debusine publish; Ubuntu codenames prepare release source, build once, gate on environment approval, then publish provenance and upload the built artifacts to S3.
qcom-preflight-checks	Security and quality gates — runs repolinter, semgrep, license checks, and dependency review.

pkg-* and qcom-* naming intentionally distinguish scope:

- pkg-* workflows are package-lifecycle workflows used by pkg-* repositories.
- qcom-* workflows are qcom-wide infrastructure/preflight workflows.

Build Routing Convention (pkg-build)

pkg-build-reusable-workflow.yml now resolves family and suite from the debian-ref branch name instead of taking a separate suite input.

Branch parsing rule:

- take the last two / -delimited fields as <family>/<suite>
- family must be debian or ubuntu

Examples:

- qcom/debian/latest (maps to sid)
- qcom/debian/bookworm
- qcom/ubuntu/resolute
- test/qcom/ubuntu/resolute
- ubuntu/resolute
- dev/whatever/yo/debian/trixie

Invalid examples:

- resolute
- ubuntu
- ubuntu-resolute

For PR jobs that build transient heads (for example debian/pr/*), workflow routing falls back to the PR base branch (github.base_ref).

Builder Images

qcom-build-utils consumes two image families:

- ghcr.io/qualcomm-linux/debusine-pkg-builder:{suite} for Debian suites routed through Debusine
- ghcr.io/qualcomm-linux/pkg-builder:{codename} for Ubuntu codenames routed through the local build path

The Debusine-specific helper scripts used by the Debian path are checked out from qualcomm-linux/debusine-action at runtime.

Composite Actions

Action	Description
build_package	Builds Debian packages using git-buildpackage and sbuild. Supports native ARM64 builds and cross-compilation.
abi_checker	Compares ABI compatibility against the previously published version using libabigail. Returns a bitmask indicating compatibility status.
push_to_repo	Uploads built .deb / .ddeb packages to the pkg-oss-staging-repo APT repository with deduplication.

Getting Started

Creating a New Package Repository

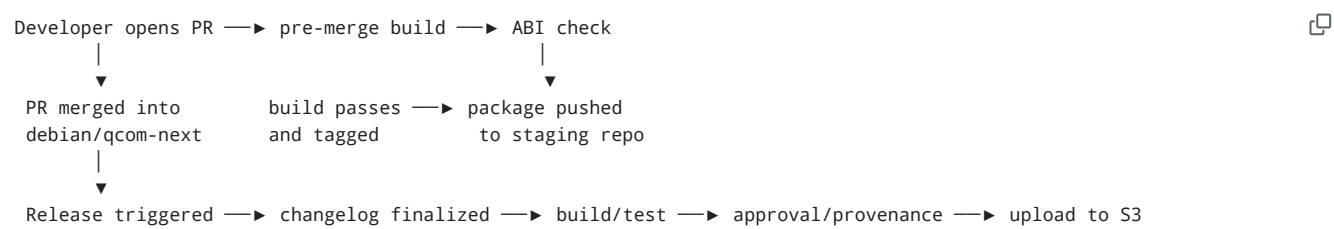
1. Use the [pkg-template](#) — click "Use this template" and name your repo with the `pkg-` prefix (e.g., `pkg-mypackage`). Enable "Include all branches".
2. Customize the `debian/` directory on the `debian/qcom-next` branch for your package.
3. Set repository variables:
 - `UPSTREAM_REPO_GITHUB_NAME` — in the **package repo**, points to the upstream source repo (e.g., `qualcomm-linux/qcom-example-package-source`).
 - `PKG_REPO_GITHUB_NAME` — in the **upstream repo**, points to the package repo (e.g., `qualcomm-linux/pkg-example`).
4. Configure branch protection for `debian/qcom-next` with the `pkg-build.yml` workflow check as a required status check.
5. Copy `.github/TO_PASTE_IN_UPSTREAM_REPO/pkg-build-pr-check.yml` into the upstream repo's `.github/workflows/` on its default branch.

See [pkg-example](#) for a complete working reference.

🔗 **Package Repo Branch Structure**

Branch	Purpose
main	Workflows, docs, and boilerplate files
debian/qcom-next	Active Debian packaging branch (build target)
debian/<version>	Version-specific tags/branches (e.g., <code>debian/1.1.0-1</code>)
upstream/latest	Latest upstream source (non-native packages)
upstream/<version>	Tagged upstream versions

🔗 **Typical Workflow Lifecycle**



1. **Pre-merge:** A PR against `debian/qcom-next` triggers a build and ABI compatibility check.
2. **Post-merge:** On merge, the package is built, pushed to the staging APT repo, and tagged `debian/<version>` .
3. **Upstream promotion:** When the upstream project tags a new release, the promote workflow merges it into the packaging branch and opens a PR.
4. **Upstream PR validation:** PRs in the upstream repo are validated against the package build to catch breakages early.
5. **Release:** A manual dispatch finalizes changelog state, builds and tests once, waits on `pkg-release-approval` , then pushes release git state, publishes provenance, uploads artifacts to S3, and notifies [qcom-distro-images](#).

🔗 **Build Infrastructure**

Component	Details
Container images	<code>ghcr.io/qualcomm-linux/debusine-pkg-builder:{suite}</code> for Debian/Debusine execution and <code>ghcr.io/qualcomm-linux/pkg-builder:{codename}</code> for Ubuntu/pkg-builder execution
Staging APT repo	pkg-oss-staging-repo served via GitHub Pages
Runners	<code>ubuntu-latest</code> for the Debian Debusine path and ARM pkg-builder runners for the local Ubuntu build/release path
Artifact storage	S3 for release builds

🔗 Build & Utility Scripts

🔗 Package Build Scripts (scripts/)

Script	Description
deb_abi_checker.py	Compares ABI between package versions using <code>libabigail</code> . Return codes: 0 no diff, 1 compatible, 2 incompatible, 4 stripped, 8 not found, 16 PPA error.
merge_debian_packaging_upstream	Merges an upstream commitish into the <code>debian/</code> branch, preserving <code>debian/</code> and <code>.github/</code> directories.
ppa_interface.py	Interfaces with APT repositories — download, list, and query package versions.
ppa_organizer.py	Organizes build output into APT pool structure.
create_promotion_pr.py	Generates PR title and body for upstream promotions.
helpers.py	Shared utilities for directory management, logging, and APT server setup.

🔗 Platform Build Scripts

Script	Description
kernel/scripts/build_kernel.sh	Builds the Linux kernel for ARM64 with Qualcomm defconfig.
kernel/scripts/build-kernel-deb.sh	Packages kernel artifacts into an Ubuntu-compliant <code>.deb</code> .
kernel/scripts/build-dtb-image.sh	Builds a FAT-formatted Device Tree Blob image for Qualcomm platforms.
bootloader/build-efi-esp.sh	Creates a deterministic EFI System Partition (<code>efi.bin</code>) for ARM64.
rootfs/scripts/build-rootfs.sh	Generates a bootable ext4 root filesystem image using <code>debootstrap</code> .

🔗 Local Package Building

For building packages locally outside of GitHub Actions, see the [docker-pkg-build](#) repository which provides containerized Debian package builds.

🔗 Documentation

Detailed documentation is available in the [docs/](#) directory:

- [Workflow Architecture](#) — system overview and component interactions
- [Reusable Workflows](#) — detailed reference for each workflow
- [GitHub Actions](#) — composite action reference and patterns
- [Package Repository Integration](#) — step-by-step setup guide

🔗 Related Repositories

Repository	Description
pkg-template	Template for creating new <code>pkg-*</code> package repositories
pkg-example	Complete working example of a package repository
qcom-example-package-source	Example upstream source repo with package build integration
docker-pkg-build	Containerized local Debian package builder
pkg-oss-staging-repo	Staging APT repository for built packages
qcom-distro-images	Distribution image configuration consuming released packages

Branches

main: Primary development branch. Contributors should develop submissions based on this branch and submit pull requests to this branch.

Contributing

See [CONTRIBUTING.md](#) for details on the branching strategy, pull request process, and DCO sign-off requirements.

Getting in Contact

- [Report an Issue on GitHub](#)
- [Open a Discussion on GitHub](#)

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors

No contributors

Languages

Shell 62.5% Python 37.5%

Suggested workflows

Based on your tech stack



Django
Build and Test a Django Project
By GitHub Actions

Configure



Pylint
Lint a Python application with pylint.
By GitHub Actions

Configure



Publish Python Package
Publish a Python Package to PyPI on release.
By GitHub Actions

Configure

[More workflows](#)

